

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 5: DIVIDE AND CONQUER

(MERGE SORT-QUICK SORT)

Abdullah Bal, PhD

Objectives



Recap

Decrease by constant factor
Variable size decrease
Lomuto's Partitioning
Quickselect
Binary search tree



Divide-and-Conquer



Master Theorem Merge Sort



Quick Sort Hoare's partitioning

Divide-and-Conquer

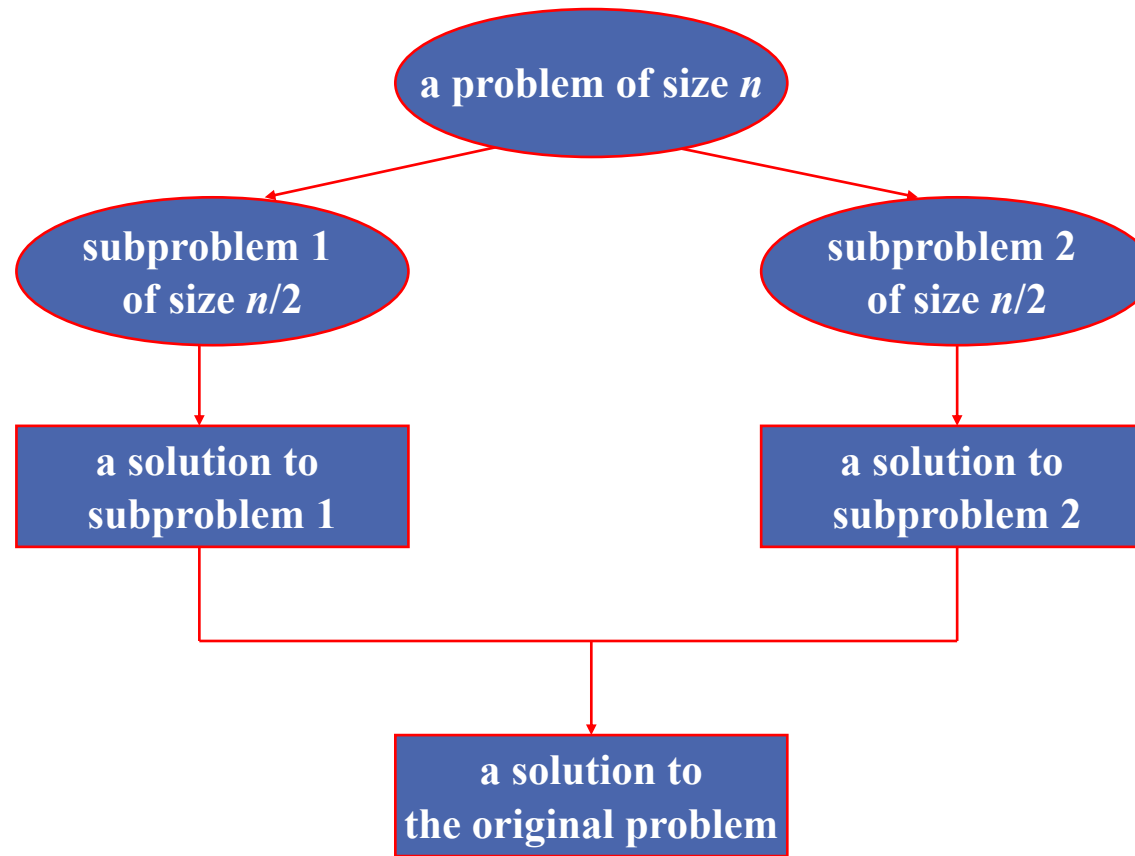
The most-well known algorithm design strategy:

1. Divide instance of problem into **two or more smaller instances**
2. Solve smaller instances recursively
3. Obtain **solution to original (larger) instance** by combining these solutions

Divide-and-Conquer

- The divide-and-conquer approach yields some of the most **important and efficient algorithms** in computer science.
- The divide-and-conquer technique is ideally suited for **parallel computations**, in which each subproblem can be solved simultaneously by its own processor.

Divide-and-Conquer Technique (cont.)



Divide-and-Conquer Examples

- Sorting: Merge Sort and Quick Sort
- Binary tree traversals
- Multiplication of large integers
- Matrix multiplication: Strassen's algorithm
- Closest-pair and convex-hull algorithms

Divide-and-Conquer Analysis

- In the most typical case of divide-and-conquer a problem's instance of size n is divided into two instances of size $n/2$.
- More generally, an instance of size n can be divided into b instances of size n/b , with a of them needing to be solved.

(a and b are constants; $a \geq 1$ and $b > 1$.)

- Assuming that size n is a power of b to simplify our analysis, we get the following recurrence for the running time $T(n)$:

$$T(n) = a.T(n/b) + f(n)$$

Divide-and-Conquer Analysis

$$T(n)=a.T(n/b)+f(n)$$

where $f(n)$ is a function that accounts for the time spent on dividing an instance of size n into instances of size n/b and combining their solutions.

- This recursive formula is called the **general divide-and-conquer recurrence**.
- $T(n)$ depends on the values of the constants a and b and the order of growth of the function $f(n)$.

Master theorem

- General divide-and-conquer recurrence formula:

$$T(n) = a.T(n/b) + f(n)$$

- The **efficiency analysis** of many divide-and-conquer algorithms is greatly **simplified by** the following **Master theorem**:

- If $f(n) \in \Theta(n^d)$ where $d \geq 0$, then

If $a < b^d$, $T(n) \in \Theta(n^d)$

If $a = b^d$, $T(n) \in \Theta(n^d \log n)$

If $a > b^d$, $T(n) \in \Theta(n^{\log_b a})$

- Analogous results hold for the O and Ω notations, too.

Master theorem Examples

Examples: $T(n) = 4T(n/2) + n \Rightarrow T(n) \in ?$

$$T(n) = 4T(n/2) + n^2 \Rightarrow T(n) \in ?$$

$$T(n) = 4T(n/2) + n^3 \Rightarrow T(n) \in ?$$

Master theorem Examples (cont.)

Solutions: $T(n) = 4T(n/2) + n \Rightarrow T(n) \in \Theta(n^2)$

$$T(n) = 4T(n/2) + n^2 \Rightarrow T(n) \in \Theta(n^2 \log n)$$

$$T(n) = 4T(n/2) + n^3 \Rightarrow T(n) \in \Theta(n^3)$$

Example: Divide-and-Conquer Analysis

- For example, the recurrence for the number of additions $A(n)$ made by the divide-and-conquer sum-computation algorithm on inputs of size $n = 2^k$:

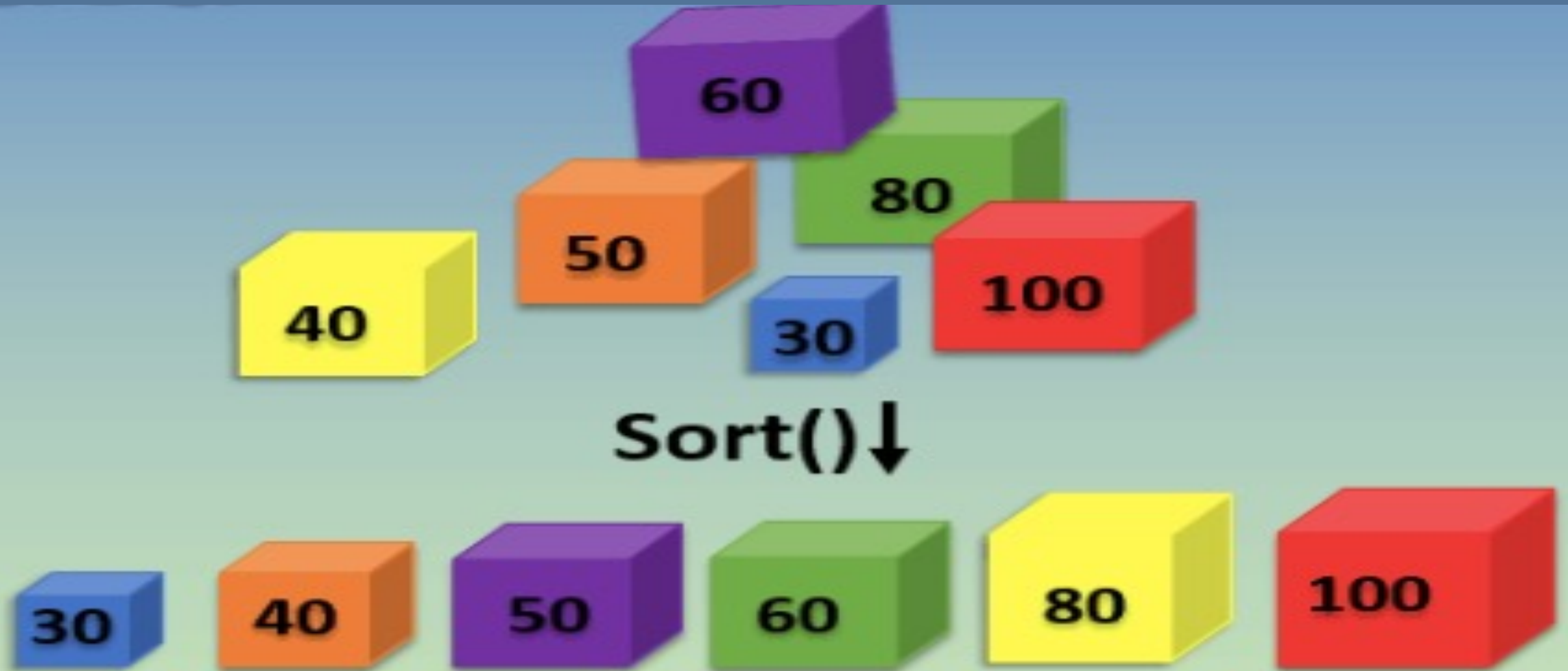
$$A(n) = 2A(n/2) + 1$$

- For this example, $a = 2$, $b = 2$, and $d = 0$; hence, since $a > b^d$,

$$A(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n).$$

- We were able to find the solution's efficiency class without going through the drudgery of solving the recurrence.

Merge Sort



Merge Sort

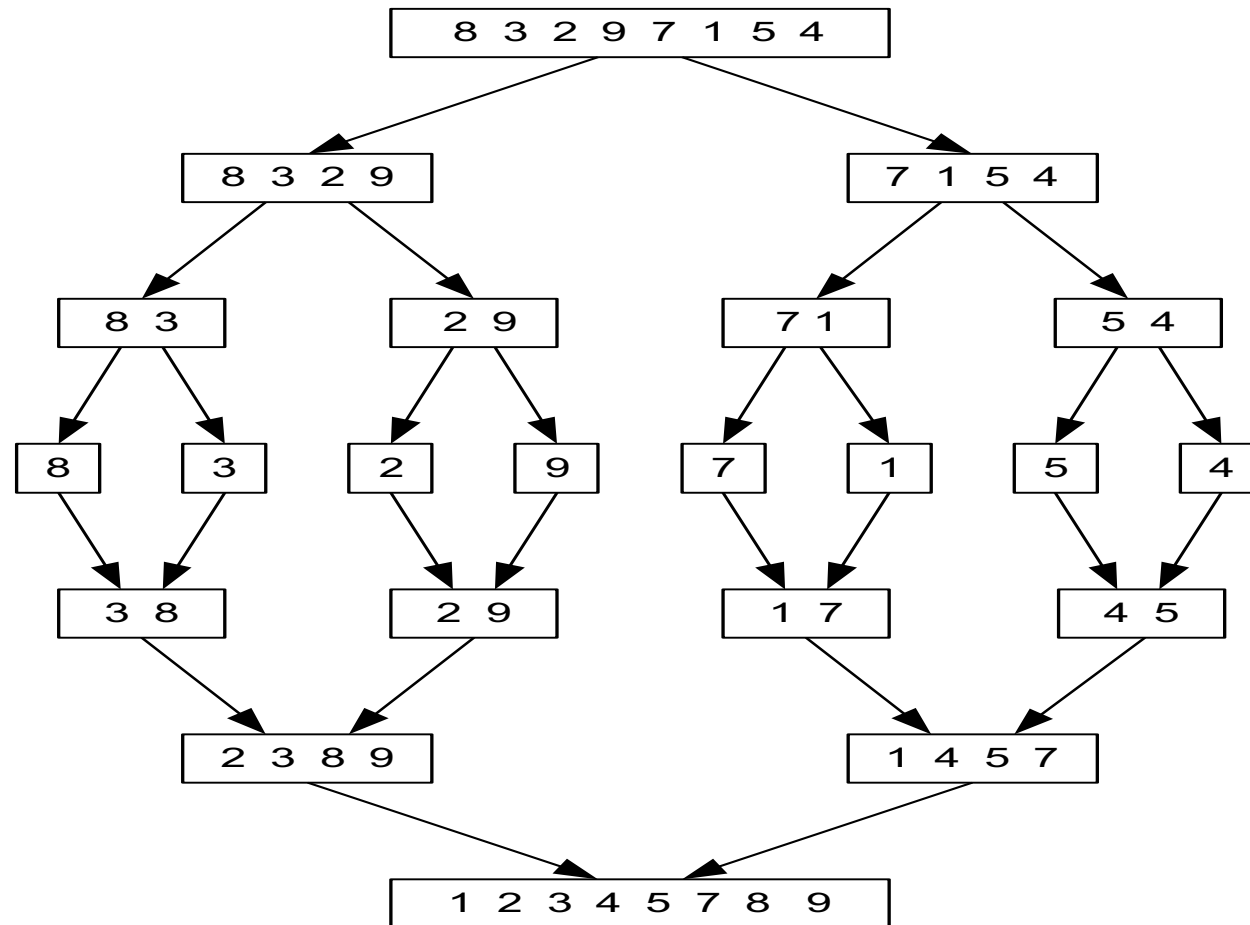
Merge Sort Steps:

- **Split array** $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C
- **Sort arrays** B and C recursively
- **Merge sorted arrays** B and C into array A as follows:
 - Repeat the following until no elements remain in one of the arrays:
 - compare the first elements in the remaining unprocessed portions of the arrays
 - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
 - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Merge Sort

6 5 3 1 8 7 2 4

Merge Sort Example



Merge Sort Example

- Apply merge sort algorithm to sort the following array.

7 5 8 2 3 9 11 10

Pseudocode of Merge Sort

ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n - 1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A)

Pseudocode of Merge

ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted

//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C

$i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow 0$

while $i < p$ **and** $j < q$ **do**

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$; $i \leftarrow i + 1$

else $A[k] \leftarrow C[j]$; $j \leftarrow j + 1$

$k \leftarrow k + 1$

if $i = p$

 copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

Analysis of Merge Sort

- Assuming for simplicity that n is a power of 2, the recurrence relation for the number of key comparisons $C(n)$ is

$$C(n) = 2C(n/2) + C_{merge}(n) \quad \text{for } n > 1, \quad C(1) = 0.$$

- Let us analyze $C_{merge}(n)$, the number of key comparisons performed during the merging stage.
- At each step, exactly one comparison is made, after which the total number of elements in the two arrays still needing to be processed is reduced by 1.

Analysis of Merge Sort

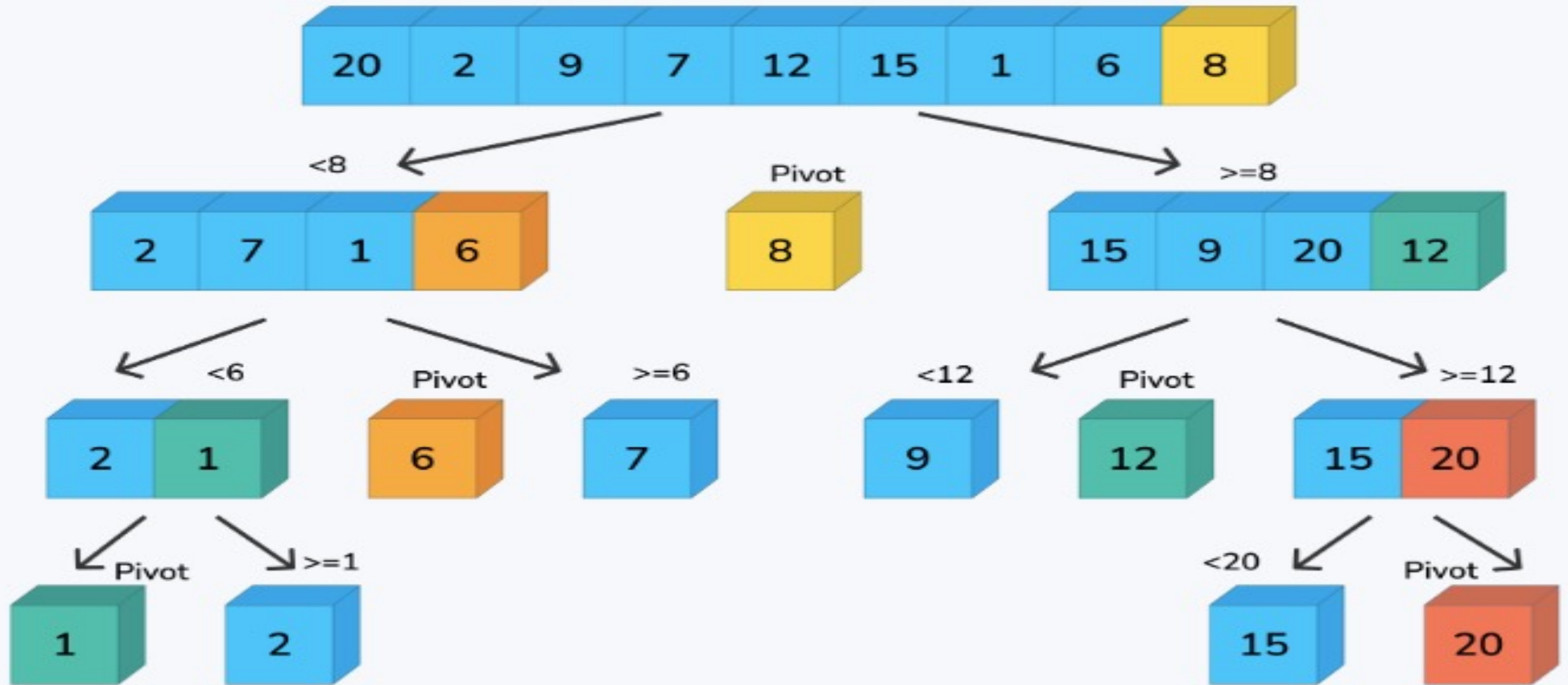
- In the worst case, neither of the two arrays becomes empty before the other one contains just one element (e.g., smaller elements may come from the alternating arrays).
- Therefore, for the worst case, $C_{\text{merge}}(n) = n - 1$, and we have the recurrence

$$C_{\text{worst}}(n) = 2C_{\text{worst}}(n/2) + n - 1 \quad \text{for } n > 1, \quad C_{\text{worst}}(1) = 0.$$

- According to the **Master Theorem**, $C_{\text{worst}}(n) \in \Theta(n \log n)$.
- It is easy to find the exact solution to the worst-case recurrence using backward substitution:

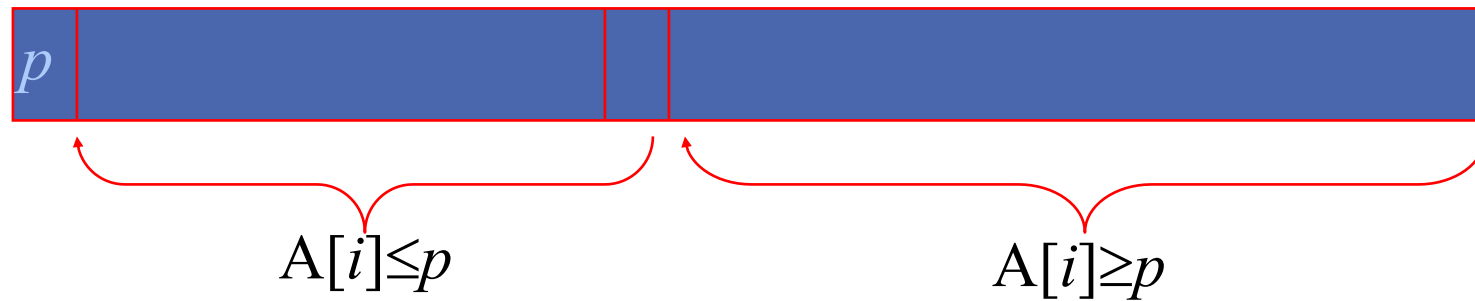
$$C_{\text{worst}}(n) = n \log_2 n - n + 1.$$

Quick Sort



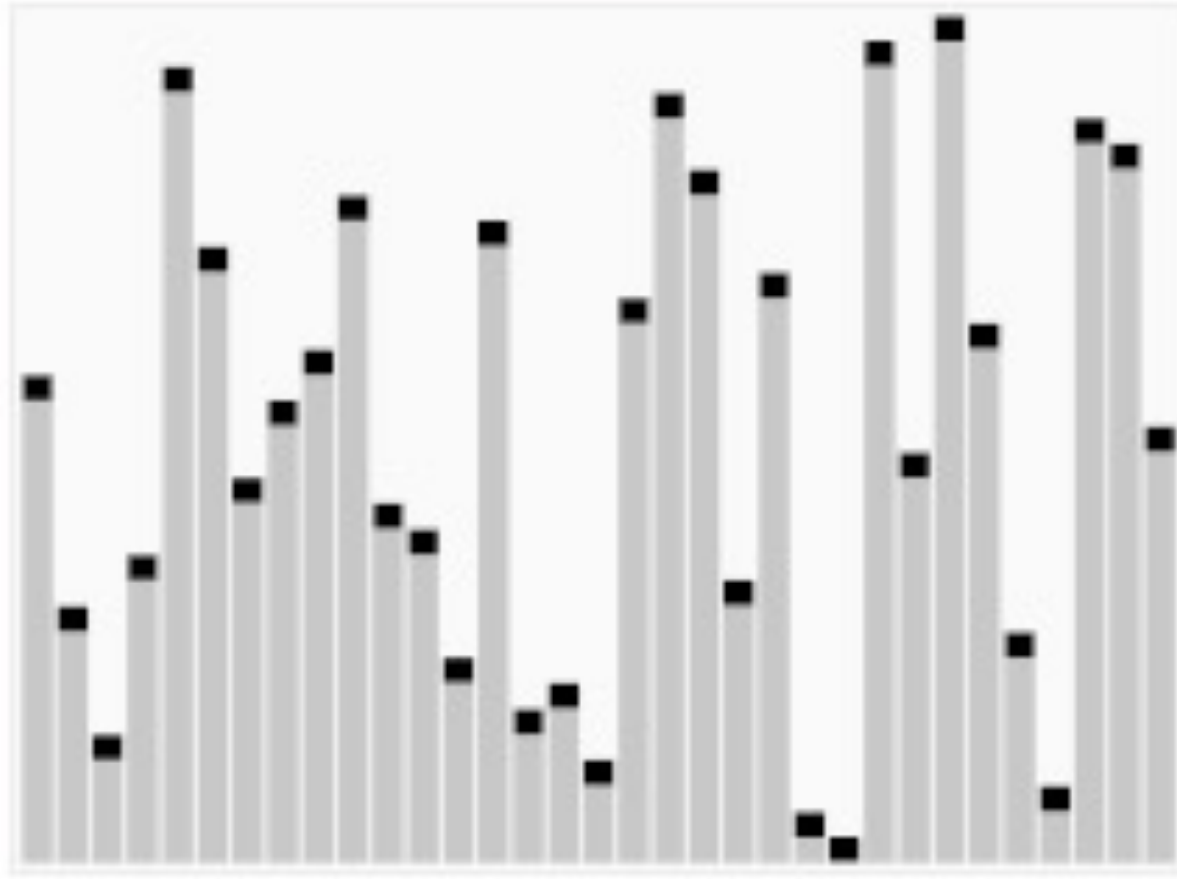
Quick Sort

- Select a *pivot* (partitioning element) – here, the first element
- Rearrange the list so that all the elements in the first s positions are smaller than or equal to the pivot and all the elements in the remaining $n-s$ positions are larger than or equal to the pivot.



- Exchange the pivot with the last element in the first (i.e., $\leq$) subarray — the pivot is now in its final position
- Sort the two subarrays recursively

Quick Sort



Quicksort-Partitioning

- Unlike **mergesort**, which divides its input elements according to their **position** in the array, **quicksort** divides them according to their **value**.
- We already encountered this idea of an array partition in the selection problem.
- A **partition is an arrangement** of the array's elements so that all the elements to the left of some element $A[s]$ are less than or equal to $A[s]$, and all the elements to the right of $A[s]$ are greater than or equal to it:

$$\underbrace{A[0] \dots A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \dots A[n-1]}_{\text{all are } \geq A[s]}$$

Quick Sort-Partitioning

- After a partition is achieved, $A[s]$ will be in its final position in the sorted array, and we can **continue sorting the two subarrays** to the left and to the right of $A[s]$ independently (e.g., by the same method).
- Difference with merge Sort:
 - **Merge Sort:** The division of the problem into two subproblems is immediate and the **entire work happens in combining their solutions**;
 - **Quick Sort:** The entire work happens in the division stage, with **no work required to combine the solutions** to the subproblems.

Quicksort

ALGORITHM *Quicksort*($A[l..r]$)

//Sorts a subarray by quicksort

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right

// indices l and r

//Output: Subarray $A[l..r]$ sorted in nondecreasing order

if $l < r$

$s \leftarrow \text{Partition}(A[l..r])$ // s is a split position

Quicksort($A[l..s - 1]$)

Quicksort($A[s + 1..r]$)

Hoare's Partitioning Algorithm

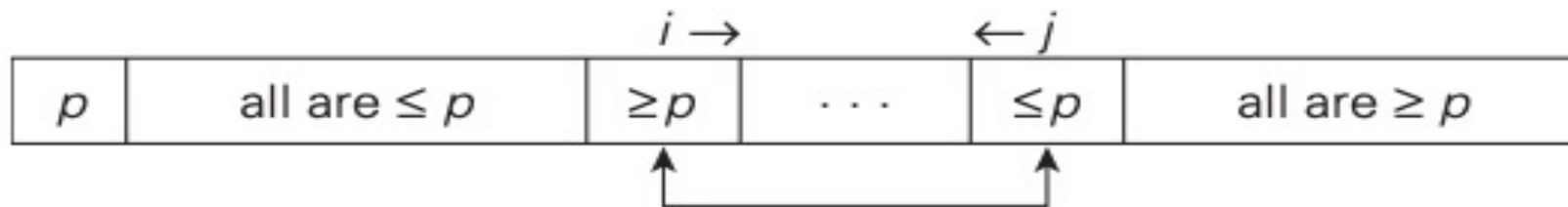
- Scan the subarray from both ends, comparing the subarray's elements to the pivot.
- The left-to-right scan, denoted by index pointer i , starts with the second element.
- Since we want elements smaller than the pivot to be in the left part of the subarray, this scan skips over elements that are smaller than the pivot and stops upon encountering the first element greater than or equal to the pivot.

Hoare's Partitioning Algorithm

- The right-to-left scan, denoted by index pointer j , starts with the last element of the subarray.
- Since we want elements larger than the pivot to be in the right part of the subarray, this scan skips over elements that are larger than the pivot and stops on encountering the first element smaller than or equal to the pivot.
- For example, if we did otherwise for an array of n equal elements, we would have gotten a split into subarrays of sizes $n - 1$ and 0 , reducing the problem size just by 1 after scanning the entire array.

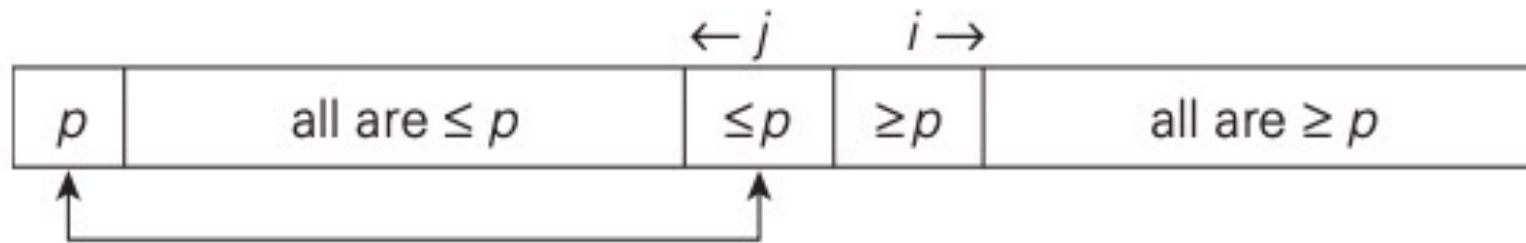
Hoare's Partitioning Algorithm

- After **both scans stop**, **three situations may arise**, depending on whether or not the scanning indices have crossed.
- 1-** If scanning indices i and j have **not crossed**, i.e., $i < j$, we simply exchange $A[i]$ and $A[j]$ and resume the scans by **incrementing i and decrementing j** , respectively:



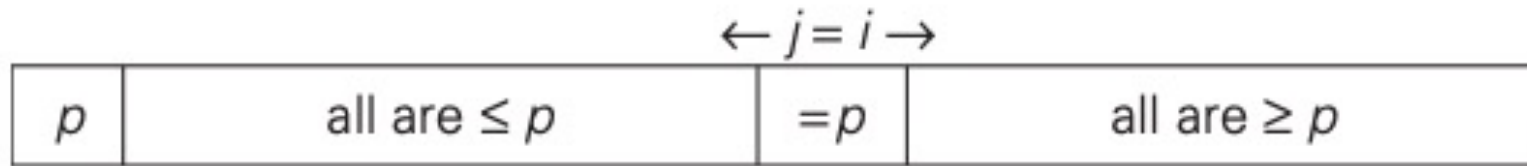
Hoare's Partitioning Algorithm

2- If the scanning indices have **crossed over**, i.e., $i > j$, we will have partitioned the subarray after **exchanging the pivot with $A[j]$** :



Hoare's Partitioning Algorithm

3- Finally, if the scanning indices stop while **pointing to the same element, i.e., $i = j$** , the value they are pointing to must be equal to p . Thus, we have the subarray partitioned, with the split position $s = i = j$



Hoare's Partitioning Algorithm

ALGORITHM *HoarePartition*($A[l..r]$)

//Partitions a subarray by Hoare's algorithm, using the first element

// as a pivot

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right

// indices l and r ($l < r$)

//Output: Partition of $A[l..r]$, with the split position returned as

// this function's value

$p \leftarrow A[l]$

$i \leftarrow l; j \leftarrow r + 1$

repeat

repeat $i \leftarrow i + 1$ **until** $A[i] \geq p$

repeat $j \leftarrow j - 1$ **until** $A[j] \leq p$

 swap($A[i]$, $A[j]$)

until $i \geq j$

swap($A[i]$, $A[j]$) //undo last swap when $i \geq j$

swap($A[l]$, $A[j]$)

return j

Quick Sort Example

- Apply quick sort algorithm to sort the following array.

5 3 1 9 8 2 4 7

Quick Sort Example

Step 1:

0	1	2	3	4	5	6	7
5	<i>i</i> 3	1	9	8	2	4	<i>j</i> 7
5	3	1	<i>i</i> 9	8	2	<i>j</i> 4	7
5	3	1	<i>i</i> 4	8	2	<i>j</i> 9	7
5	3	1	4	<i>i</i> 8	<i>j</i> 2	9	7
5	3	1	4	<i>i</i> 2	<i>j</i> 8	9	7
5	3	1	4	<i>j</i> 2	<i>i</i> 8	9	7
2	3	1	4	5	8	9	7

Quick Sort Example

Step 2: 2 3 1 4 **5** 8 9 7

	<i>i</i>		<i>j</i>
2	3	1	4
2	<i>i</i> 3	<i>j</i> 1	4
2	<i>i</i> 1	<i>j</i> 3	4
2	<i>j</i> 1	<i>i</i> 3	4
1	2	3	4
1			

Quick Sort Example

Step 3:

1

2

3

4

3

j
3

i j

4

i
4

Quick Sort Example

Step 4: 2 3 1 4 **5** 8 9 7

	<i>i</i>	<i>j</i>
8	9	7
8	<i>i</i> 7	<i>j</i> 9
8	<i>j</i> 7	<i>i</i> 9
7	8	9
7		
		9

Quick Sort Example

0	1	2	3	4	5	6	7
5	<i>i</i> 3	1	9	8	2	4	<i>j</i> 7
5	3	1	<i>i</i> 9	8	2	<i>j</i> 4	7
5	3	1	<i>i</i> 4	8	2	<i>j</i> 9	7
5	3	1	4	<i>i</i> 8	<i>j</i> 2	9	7
5	3	1	4	<i>i</i> 2	<i>j</i> 8	9	7
5	3	1	4	<i>j</i> 2	<i>i</i> 8	9	7
2	3	1	4	5	8	9	7
2	<i>i</i> 3	1	<i>j</i> 4				
2	<i>i</i> 3	<i>j</i> 1	4				
2	<i>i</i> 1	<i>j</i> 3	4				
2	<i>j</i> 1	<i>i</i> 3	4				
1	2	3	4				
1		3	<i>i j</i> 4				
		<i>j</i> 3	<i>i</i> 4				
			4				
					8	<i>i</i> 9	<i>j</i> 7
					8	<i>i</i> 7	<i>j</i> 9
					8	<i>j</i> 7	<i>i</i> 9
					7	8	9
					7		9

Result: 1 2 3 4 5 7 8 9

Quick Sort Example

- Apply quick sort algorithm to sort the following array.

7 5 8 2 3 9 11 10 4

Analysis of Quick Sort

- Best case: Split in the middle — $\Theta(n \log n)$
- Worst case: Sorted array! — $\Theta(n^2)$
- Average case: Random arrays — $\Theta(n \log n)$
- Improvements:
 - Better pivot selection methods such as *randomized quicksort* that uses a random element or the *median-of-three* method that uses the median of the leftmost, rightmost, and the middle element of the array
 - Switching to insertion sort on very small subarrays (between 5 and 15 elements for most computer systems) or not sorting small subarrays at all and finishing the algorithm with insertion sort applied to the entire nearly sorted array
 - These combine to 20-25% improvement
- Considered the method of choice for internal sorting of large files ($n \geq 10,000$)

Summary

- Divide-and-Conquer
- Master Theorem
- Merge Sort
- Quick Sort
- Hoare's Partitioning

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.